

Assessment Solutions Document

Course: Design and Analysis of Algorithms (CSA06)

Q1. Knapsack - Industry Resource Allocation

Problem Statement: A cloud service provider must allocate limited server memory (100 GB) among multiple client applications, each having different memory requirements and revenue values. The goal is to maximize total revenue using the 0/1 Knapsack algorithm.

Analysis & Solution Design

The problem can be modeled as a classic 0/1 Knapsack problem where the total capacity $W = 100$ GB. The "items" are the applications, the "weights" are their memory requirements, and the "values" are their revenues.

- **Dynamic Programming Approach:** We use a 2D array or table where $dp[i][w]$ represents the maximum revenue achievable using a subset of the first i applications with a total memory limit of w .
- **Recurrence Relation:** $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$
- **Optimal Substructure & Overlapping Subproblems:** The optimal substructure is utilized because the maximum revenue for capacity w using i applications relies on the optimal solutions to smaller subproblems (either including or excluding the i -th application). Overlapping subproblems are addressed by storing these intermediate maximum revenues in the DP table to avoid redundant recalculations.

Implementation & Performance

- **Tools/Platforms:** Python or C++ are highly suitable due to their efficient array and matrix handling capabilities.
- **Complexity:** The time complexity is $O(N \times W)$ and the space complexity is $O(N \times W)$ (which can be optimized to $O(W)$ using a 1D array), where N is the number of applications and W is the memory capacity. In an industry context, this pseudo-polynomial time is highly efficient for fixed capacities like 100 GB.

Q5. Dijkstra - Navigation System

Problem Statement: A navigation system computes shortest paths between locations in real-time. The task is to apply Dijkstra's algorithm to solve this efficiently.

Analysis & Solution Design

Dijkstra's algorithm is a greedy algorithm that solves the single-source shortest path problem for a graph with non-negative edge weights.

- **Step-by-Step Execution:**

1. Initialize the distance to the starting location as 0 and all other locations as ∞ (infinity).
2. Insert all locations into a Priority Queue based on their current shortest distance.
3. Extract the location with the minimum distance.
4. Iterate through its neighboring locations. If the current node's distance plus the edge weight to the neighbor is less than the neighbor's known distance, update the neighbor's distance and adjust its position in the queue.

- **Data Structures:** A Min-Priority Queue (often implemented via a binary heap or Fibonacci heap) is essential to efficiently fetch the next closest location. An adjacency list is best for representing the graph to save memory.

Implementation & Performance

- **Tools/Platforms:** C++ using `std::priority_queue` or Python using `heapq`. Graph databases like Neo4j are also useful for real-world mapping.
- **Complexity:** Using an adjacency list and a binary heap priority queue, the time complexity is $O((V + E) \log V)$, where V is vertices (locations) and E is edges (roads). This ensures low-latency route calculations necessary for real-world GPS navigation systems.

Q6. Huffman Coding - Data Compression

Problem Statement: A video streaming platform compresses data based on symbol frequencies to optimize storage and transmission. The task is to use Huffman coding to generate optimal prefix codes.

Analysis & Solution Design

Huffman coding is a greedy algorithm used for lossless data compression. It assigns variable-length codes to input characters, with lengths based on the frequencies of the corresponding characters.

- **Tree Construction:**

1. Calculate the frequency of each symbol in the data.
2. Create a leaf node for each symbol and build a Min-Heap of all nodes.
3. Extract the two nodes with the lowest frequencies. Create a new internal node with a frequency equal to the sum of the two nodes.
4. Make the first extracted node the left child and the second the right child. Insert this new node back into the Min-Heap.
5. Repeat until the heap contains only one node (the root of the Huffman tree).

- **Encoding Process:** Traverse the tree from root to leaf. Assign '0' for left edges and '1' for right edges. The path to each leaf dictates its prefix code.

Implementation & Performance

- **Tools/Platforms:** Standard C/C++ libraries or Python can be used to implement the tree. Real-world tools include standard compression libraries like zlib.
- **Complexity:** The time complexity is $O(N \log N)$ where N is the number of unique symbols, dictated by the heap operations. This results in significant bandwidth and storage optimization by reducing the average number of bits per symbol.

Q8. Activity Selection - Scheduling Optimization

Problem Statement: A software company must schedule multiple client meetings in limited time slots, ensuring the maximum number of non-overlapping meetings. This Activity Selection problem is to be solved using a Greedy approach.

Analysis & Solution Design

The Activity Selection problem asks for the maximum size set of mutually compatible activities.

- **Selection Strategy:**

1. Sort all available meetings based on their finishing times in ascending order.
2. Select the first meeting from the sorted list.
3. Iterate through the remaining meetings. If a meeting's start time is greater than or equal to the finish time of the previously selected meeting, select it.

- **Greedy Justification:** This greedy method provides an optimal solution because choosing the activity that finishes first leaves the maximum possible remaining time for other activities to be scheduled.

Implementation & Performance

- **Tools/Platforms:** Java or Python, utilizing built-in sorting functions with custom comparators for the finish times.
- **Complexity:** Sorting the activities takes $O(N \log N)$ time, and the linear scan to select activities takes $O(N)$ time. The overall time complexity is $O(N \log N)$. This ensures highly efficient scheduling for daily business operations.

Q9. Coin Change - Payment System Optimization

Problem Statement: A digital payment system must return change to users using the minimum number of coins from available denominations. The problem is to be solved using Dynamic Programming and compared with a Greedy approach.

Analysis & Solution Design

- **Dynamic Programming (DP) Solution:** We build a table dp where $dp[i]$ represents the minimum number of coins needed to make amount i . We initialize $dp[0] = 0$ and all other indices to ∞ (infinity).
- **Recurrence Relation:** For each amount a from 1 to the target, and for each available coin denomination c :
 $dp[a] = \min(dp[a], dp[a - c] + 1)$ (provided $a \geq c$).
- **Comparison with Greedy:** A greedy approach (picking the largest possible coin first) works for canonical coin systems (like US currency: 1, 5, 10, 25). However, it fails for arbitrary denominations. For example, if coins are {1, 3, 4} and the target is 6, Greedy gives 4, 1, 1 (3 coins), while DP finds the optimal 3, 3 (2 coins). DP is applicable universally when optimal substructure holds.

Implementation & Performance

- **Tools/Platforms:** Python, Java, or C++ are standard for implementing 1D array DP tables.
- **Complexity:** The time complexity for the DP approach is $O(N \times A)$, where N is the number of coin denominations and A is the total change amount. The space complexity is $O(A)$. This guarantees accuracy and systemic efficiency in financial transaction systems.